I Background

Overview of Some Deep Learning Libraries

1

Machine learning is a broad topic. Deep learning, in particular, is a way of using neural networks for machine learning. A neural network is probably a concept older than machine learning, dating back to the 1950s. Unsurprisingly, there were many libraries created for it.

The following aims to give an overview of some of the famous libraries for neural networks and deep learning. After finishing this chapter, you will learn

- ▷ Some of the deep learning or neural network libraries
- ▷ The functional difference between two common libraries, PyTorch and TensorFlow

Let's get started.

Overview

This chapter is in three parts; they are:

- ▷ The C++ Libraries
- ▷ Python Libraries
- ▷ PyTorch and TensorFlow

1.1 The C++ Libraries

Deep learning has gained attention in the last decade. Before that, there was little confident in how to train a neural network with many layers. However, understanding how to build a multilayer perceptron was around for many years.

Before we had deep learning, probably the most famous neural network library was `libann`. It is a library for C++, and the functionality is limited due to its age. This library has since stopped development. A newer library for C++ is OpenNN, which allows modern C++ syntax.

But that's pretty much all for C++. The rigid syntax of C++ may be why we do not have too many libraries for deep learning. The training phase of a deep learning project is

about experiments. We want some tools that allow us to iterate faster. Hence a dynamic programming language could be a better fit. Therefore, you see Python come on the scene.

1.2 Python Libraries

One of the earliest libraries for deep learning is Caffe. It was developed at U.C. Berkeley specifically for computer vision problems. While it is developed in C++, it serves as a library with a Python interface. Hence we can build our project in Python with the network defined in a JSON-like syntax.

Chainer is another library in Python. It is an influential one because the syntax makes a lot of sense. While it is less common nowadays, the API in Keras and PyTorch bears a resemblence to Chainer. The following is an example from Chainer's documentation, and you may mistake it as Keras or PyTorch:

```
import chainer
import chainer.functions as F
import chainer.links as L
from chainer import iterators, optimizer, training, Chain
from chainer.datasets import mnist

train, test = mnist.get_mnist()
batchsize = 128
max_epoch = 10

train_iter = iterators.SerialIterator(train, batchsize)

class MLP(Chain):
    def __init__(self, n_mid_units=100, n_out=10):
        super(MLP, self).__init__()
        with self.init_scope():
            self.l1 = L.Linear(None, n_mid_units)
            self.l2 = L.Linear(None, n_mid_units)
            self.l3 = L.Linear(None, n_out)

    def forward(self, x):
        h1 = F.relu(self.l1(x))
        h2 = F.relu(self.l2(h1))
        return self.l3(h2)

# create model
model = MLP()
model = L.Classifier(model)  # using softmax cross entropy

# set up optimizer
optimizer = optimizers.MomentumSGD()
optimizer.setup(model)

# connect train iterator and optimizer to an updater
updater = training.updaters.StandardUpdater(train_iter, optimizer)
```

```
# set up trainer and run
trainer = training.Trainer(updater, (max_epoch, 'epoch'), out='mnist_result')
trainer.run()
```

Listing 1.1: Example Chainer code

The other obsoleted library is Theano. It has ceased development, but once upon a time, it was a major library for deep learning. The earlier version of the Keras library allows you to choose between a Theano or TensorFlow backend. Indeed, neither Theano nor TensorFlow are deep learning libraries precisely. Rather, they are tensor libraries that make matrix operations and differentiation handy, upon where deep learning operations can be built. Hence these two are considered replacements for each other from Keras' perspective.

CNTK from Microsoft and Apache MXNet are two other libraries that worth mentioning. They are large with interfaces for multiple languages. Python, of course, is one of them. CNTK has C♯ and C++ interfaces, while MXNet provides interfaces for Java, Scala, R, Julia, C++, Clojure, and Perl. But recently, Microsoft decided to stop developing CNTK. MXNet does have some momentum, and it is probably the most popular library after TensorFlow and PyTorch.

Below is an example of using MXNet via the R interface. Conceptually, you see the syntax is similar to Keras' functional API:

```
require(mxnet)

train <- read.csv('data/train.csv', header=TRUE)
train <- data.matrix(train)
train.x <- train[,-1]
train.y <- train[,1]
train.x <- t(train.x/255)

data <- mx.symbol.Variable("data")
fc1 <- mx.symbol.FullyConnected(data, name="fc1", num_hidden=128)
act1 <- mx.symbol.Activation(fc1, name="relu1", act_type="relu")
fc2 <- mx.symbol.FullyConnected(act1, name="fc2", num_hidden=64)
act2 <- mx.symbol.Activation(fc2, name="relu2", act_type="relu")
fc3 <- mx.symbol.FullyConnected(act2, name="fc3", num_hidden=10)
softmax <- mx.symbol.SoftmaxOutput(fc3, name="sm")

devices <- mx.cpu()
mx.set.seed(0)
model <- mx.model.FeedForward.create(softmax, X=train.x, y=train.y,
                                     ctx=devices, num.round=10, array.batch.size=100,
                                     learning.rate=0.07, momentum=0.9,
                                     eval.metric=mx.metric.accuracy,
                                     initializer=mx.init.uniform(0.07),
                                     epoch.end.callback=mx.callback.log.train.metric(100))
```

Listing 1.2: Example MXNet code in R

1.3 PyTorch and TensorFlow

PyTorch and TensorFlow are the two major libraries nowadays. In the past, when TensorFlow was in version 1.x, they were vastly different. But since TensorFlow absorbed Keras as part of its library, these two libraries mostly work similarly.

PyTorch is backed by Facebook, and its syntax has been stable over the years. There are also a lot of existing models that we can borrow. The common way of defining a deep learning model in PyTorch is to create a class:

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class Model(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 6, kernel_size=(5,5), stride=1, padding=2)
        self.pool1 = nn.AvgPool2d(kernel_size=2, stride=2)
        self.conv2 = nn.Conv2d(6, 16, kernel_size=5, stride=1, padding=0)
        self.pool2 = nn.AvgPool2d(kernel_size=2, stride=2)
        self.conv3 = nn.Conv2d(16, 120, kernel_size=5, stride=1, padding=0)
        self.flatten = nn.Flatten()
        self.linear4 = nn.Linear(120, 84)
        self.linear5 = nn.Linear(84, 10)
        self.softmax = nn.LogSoftMax(dim=1)

    def forward(self, x):
        x = F.tanh(self.conv1(x))
        x = self.pool1(x)
        x = F.tanh(self.conv2(x))
        x = self.pool2(x)
        x = F.tanh(self.conv3(x))
        x = self.flatten(x)
        x = F.tanh(self.linear4(x))
        x = self.linear5(x)
        return self.softmax(x)

model = Model()
```

Listing 1.3: Example PyTorch code using sub-classing syntax

But there is also a sequential syntax to make the code more concise:

```
import torch
import torch.nn as nn

model = nn.Sequential(
    # assume input 1x28x28
    nn.Conv2d(1, 6, kernel_size=(5,5), stride=1, padding=2),
    nn.Tanh(),
    nn.AvgPool2d(kernel_size=2, stride=2),
    nn.Conv2d(6, 16, kernel_size=5, stride=1, padding=0),
```

```
    nn.Tanh(),
    nn.AvgPool2d(kernel_size=2, stride=2),
    nn.Conv2d(16, 120, kernel_size=5, stride=1, padding=0),
    nn.Tanh(),
    nn.Flatten(),
    nn.Linear(120, 84),
    nn.Tanh(),
    nn.Linear(84, 10),
    nn.LogSoftmax(dim=1)
)
```

Listing 1.4: Example PyTorch code using sequential syntax

TensorFlow in version 2.x adopted Keras as part of its libraries. In the past, these two were separate projects. In TensorFlow 1.x, we need to build a computation graph, set up a session, and derive gradients from a session for the deep learning model. Hence it is a bit too verbose. Keras is designed as a library to hide all these low-level details.

The same network as above can be produced by TensorFlow's Keras syntax as follows:

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, Dense, AveragePooling2D, Flatten

model = Sequential([
    Conv2D(6, (5,5), input_shape=(28,28,1), padding="same", activation="tanh"),
    AveragePooling2D((2,2), strides=2),
    Conv2D(16, (5,5), activation="tanh"),
    AveragePooling2D((2,2), strides=2),
    Conv2D(120, (5,5), activation="tanh"),
    Flatten(),
    Dense(84, activation="tanh"),
    Dense(10, activation="softmax")
])
```

Listing 1.5: Example TensorFlow code using sequential syntax

One major difference between PyTorch and Keras syntax is in the training loop. In Keras, we just need to assign the loss function, the optimization algorithm, the dataset, and some other parameters to the model. Then we have a `fit()` function to do all the training work, as follows:

```
...
model.compile(loss="categorical_crossentropy", optimizer="adam", metrics=["accuracy"])
model.fit(X_train, y_train, validation_data=(X_test, y_test), epochs=100, batch_size=32)
```

Listing 1.6: Usign `fit()` function in TensorFlow

But in PyTorch, we need to write our own training loop code:

```
# self-defined training loop function
def training_loop(model, optimizer, loss_fn, train_loader, val_loader=None, n_epochs=100):
    best_loss, best_epoch = np.inf, -1
    best_state = model.state_dict()
```

```
    for epoch in range(n_epochs):
        # Training
        model.train()
        train_loss = 0
        for data, target in train_loader:
            output = model(data)
            loss = loss_fn(output, target)
            optimizer.zero_grad()
            loss.backward()
            optimizer.step()
            train_loss += loss.item()
        # Validation
        model.eval()
        status = (f"{str(datetime.datetime.now())} End of epoch {epoch}, "
                  f"training loss={train_loss/len(train_loader)}")
        if val_loader:
            val_loss = 0
            for data, target in val_loader:
                output = model(data)
                loss = loss_fn(output, target)
                val_loss += loss.item()
            status += f", validation loss={val_loss/len(val_loader)}"
        print(status)

optimizer = optim.Adam(model.parameters())
criterion = nn.NLLLoss()
training_loop(model, optimizer, criterion, train_loader, test_loader, n_epochs=100)
```

Listing 1.7: User-defined training loop function for PyTorch

This may not be an issue if you're experimenting with a new design of network in which you want to have more control over how the loss is calculated and how the optimizer updates the model weights. But otherwise, you will appreciate the simpler syntax from Keras.

Note that both PyTorch and TensorFlow are libraries with a Python interface. Therefore, it is possible to have an interface for other languages too. For example, there are Torch for R and TensorFlow for R.

Also, note that the libraries we mentioned above are full-featured libraries that include training and prediction. If you consider a production environment where you make use of a trained model, there could be a wider choice. TensorFlow has a "TensorFlow Lite" counterpart that allows a trained model to be run on a mobile device or the web. Intel also has an OpenVINO library that aims to optimize the performance in prediction.

1.4 Further Reading

Below are the links to the libraries we mentioned above:

libann.
https://www.nongnu.org/libann/

OpenNN.
https://www.opennn.net/

Chainer.
https://chainer.org/
Caffe.
https://caffe.berkeleyvision.org/
CNTK.
https://docs.microsoft.com/en-us/cognitive-toolkit/
MXNet.
https://mxnet.apache.org/versions/1.9.1/
Theano.
https://github.com/Theano/Theano
PyTorch.
https://pytorch.org/
TensorFlow.
https://www.tensorflow.org/
Torch for R.
https://torch.mlverse.org/
TensorFlow for R.
https://tensorflow.rstudio.com/

1.5 Summary

In this chapter, you discovered various deep learning libraries and some of their characteristics. Specifically, you learned:

- ▷ What are the libraries available for C++ and Python
- ▷ How the Chainer library influenced the syntax in building a deep learning model
- ▷ The relationship between Keras and TensorFlow 2.x
- ▷ What are the differences between PyTorch and TensorFlow

In the next chapter, you will learn more about the library of choice in this book, TensorFlow.

Introduction to TensorFlow

2

TensorFlow is a Python library for fast numerical computing created and released by Google. It is a foundation library that can be used to create Deep Learning models directly or by using wrapper libraries that simplify the process built on top of TensorFlow. After completing this chapter, you will know:

- About the TensorFlow library for Python
- How to define, compile, and evaluate a simple symbolic expression in TensorFlow
- where to go to get more information on the library

Let's get started.

Overview

This chapter is in four parts; they are:

- What is TensorFlow?
- How to Install TensorFlow
- Your First Examples in TensorFlow
- More Deep Learning Models

2.1 What Is TensorFlow?

TensorFlow is an open-source library for fast numerical computing. It was created and is maintained by Google and was released under the Apache 2.0 open source license. The API is nominally for the Python programming language, although there is access to the underlying C++ API. Unlike other numerical libraries intended for use in Deep Learning like Theano, TensorFlow was designed for use both in research and development and in production systems, not least of which is RankBrain in Google search[1] and the fun DeepDream project[2]. It can run on single CPU systems and GPUs, as well as mobile devices and large-scale distributed systems of hundreds of machines.

[1] https://en.wikipedia.org/wiki/RankBrain
[2] https://en.wikipedia.org/wiki/DeepDream

2.2 How to Install TensorFlow

Installation of TensorFlow is straightforward if you already have a Python environment. If you need help setting up your workstation, see Appendix B.

TensorFlow works with Python 3.3+. You can follow the Download and Setup instructions on the TensorFlow website. Installation is probably simplest via PyPI, and specific instructions of the `pip` command to use for your Linux or Mac OS X platform are on the Download and Setup webpage. Usually, you just need to enter the following in your command line:

```
pip install tensorflow
```

An exception would be on the newer Mac with an Apple Silicon CPU. The package name for this specific architecture is `tensorflow-macos` instead:

```
pip install tensorflow-macos
```

There are also virtualenv and docker images that you can use if you prefer. To make use of the GPU, you need to have the CUDA Toolkit installed as well.

2.3 Your First Examples in TensorFlow

Computation in TensorFlow is described in terms of data flow and operations in the structure of a directed graph. The figure below is an example of how a mathematical expression can be represented.

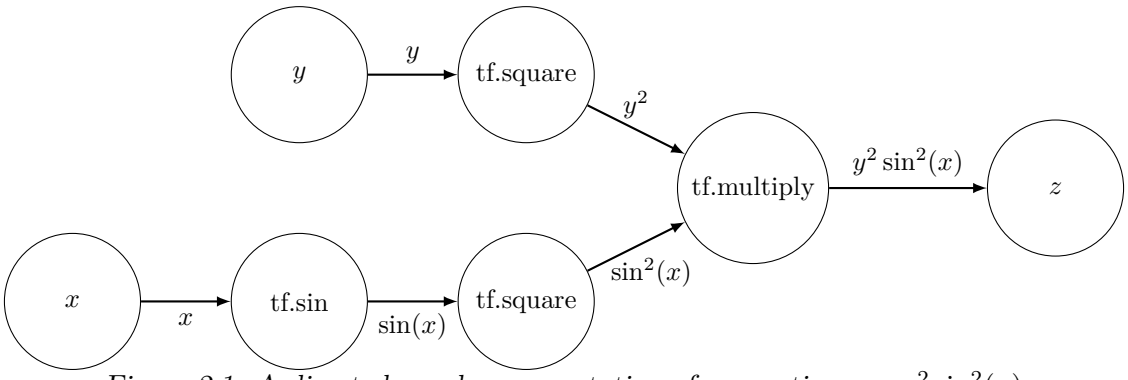

Figure 2.1: A directed graph representation of computing $z = y^2 \sin^2(x)$

There are some terms in describing a directed graph:

- ▷ **Nodes:** Nodes perform computation and have zero or more inputs and outputs. Data that moves between nodes are known as tensors, which are multi-dimensional arrays of real values.
- ▷ **Edges:** The graph defines the flow of data, branching, looping, and updates to state. Special edges can be used to synchronize behavior within the graph, for example, waiting for computation on a number of inputs to complete.

▷ **Operation:** An operation is a named abstract computation that can take input attributes and produce output attributes. For example, you could define an add or multiply operation.

We seldom use these terms when we write TensorFlow programs. But understanding them helps figuring out what TensorFlow does to our code.

Computation with TensorFlow

This first example is a modified version of an example on TensorFlow website. It shows how you can define values as *tensors* and execute an operation.

```
import tensorflow as tf
a = tf.constant(10)
b = tf.constant(32)
print(a+b)
```

Listing 2.1: A simple TensorFlow program for addition

Running this example displays:

```
tf.Tensor(42, shape=(), dtype=int32)
```

Output 2.1: Result of TensorFlow addition

Linear Regression with TensorFlow

This next example comes from the introduction in the older version of TensorFlow tutorial.

This example shows how you can define variables (e.g., `W` and `b`) as well as variables resulting from computation (`y`). Below, there is automatic differentiation under the hood. When we use `mse_loss()` function to compute the differences between `y` and `y_data`, there is a graph created connecting the values produced by the function (`loss`) to the TensorFlow variables `W` and `b`. TensorFlow uses this graph to deduce how to update the variables inside the `minimize()` function.

```
import tensorflow as tf
import numpy as np

# Create 100 phony x, y data points in NumPy, y = x * 0.1 + 0.3
x_data = np.random.rand(100).astype(np.float32)
y_data = x_data * 0.1 + 0.3

# Try to find values for W and b that compute y_data = W * x_data + b
# (We know that W should be 0.1 and b 0.3, but Tensorflow will figure that out for us.)
W = tf.Variable(tf.random.normal([1]))
b = tf.Variable(tf.zeros([1]))

# A function to compute mean squared error between y_data and computed y
def mse_loss():
    y = W * x_data + b
```

```
    loss = tf.reduce_mean(tf.square(y - y_data))
    return loss

# Minimize the mean squared errors.
optimizer = tf.keras.optimizers.Adam()
for step in range(5000):
    optimizer.minimize(mse_loss, var_list=[W,b])
    if step % 500 == 0:
        print(step, W.numpy(), b.numpy())

# Learns best fit is W: [0.1], b: [0.3]
```

Listing 2.2: Linear regression in TensorFlow

Running this example prints the following output:

```
0 [-0.35913563] [0.001]
500 [-0.04056413] [0.3131764]
1000 [0.01548613] [0.3467598]
1500 [0.03492216] [0.3369852]
2000 [0.05408324] [0.32609695]
2500 [0.07121297] [0.316361]
3000 [0.08443557] [0.30884594]
3500 [0.09302785] [0.3039626]
4000 [0.09754606] [0.3013947]
4500 [0.09936733] [0.3003596]
```

Output 2.2: Progress of running linear regression using TensorFlow

You can learn more about the mechanics of TensorFlow in its Basic Usage guide.

2.4 More Deep Learning Models

Your TensorFlow installation comes with a number of Deep Learning models that you can use and experiment with directly. Firstly, you need to find out where TensorFlow was installed on your system. For example, you can use the following Python script on the command line:

```
python -c 'import os,tensorflow;print(os.path.dirname(tensorflow.__file__))'
```

Listing 2.3: Command to print TensorFlow install location

For example, this could be:

```
/usr/lib/python3.9/site-packages/tensorflow
```

Output 2.3: TensorFlow install location

Under the subdirectories of this location, you will find a number of deep learning models with detailed comments, such as:

- ▷ Multi-threaded word2vec mini-batched skip-gram model
- ▷ Multi-threaded word2vec unbatched skip-gram model

- ▷ CNN for the CIFAR-10 network
- ▷ Simple, end-to-end, LeNet-5-like convolutional MNIST model example
- ▷ Sequence-to-sequence model with an attention mechanism

Also, check the examples directory, which contains an example using the MNIST dataset.

There is also an excellent list of tutorials on the main TensorFlow website. They show how to use different neural network constructions and different datasets, and how to use the framework in various ways.

Finally, there is the TensorFlow playground where you can experiment with small neural network models right in your web browser.

2.5 TensorFlow Resources

Below are the resources mentioned in this chapter:

TensorFlow.
https://www.tensorflow.org/
Install TensorFlow 2.
https://www.tensorflow.org/install
TensorFlow project on GitHub.
https://github.com/tensorflow/tensorflow
TensorFlow tutorials.
https://www.tensorflow.org/tutorials
TensorFlow Basic Usage Guide.
https://www.tensorflow.org/guide/basics
TensorFlow Playground.
https://playground.tensorflow.org/

2.6 Summary

In this chapter, you discovered the TensorFlow Python library for deep learning.

You learned that it is a library for fast numerical computation, specifically designed for the types of operations required to develop and evaluate of large deep learning models.

In the next chapter, you will learn about the autograd function from TensorFlow and how it became the basis of deep learning training.

Using Autograd in TensorFlow to Solve a Regression Problem

3

We usually use TensorFlow to build a neural network. However, TensorFlow is not limited to this. Behind the scenes, TensorFlow is a tensor library with automatic differentiation capability. Hence you can easily use it to solve a numerical optimization problem with gradient descent, which is the algorithm to train a neural network. In this chapter, you will learn how TensorFlow's automatic differentiation engine, autograd, works. After finishing this chapter, you will learn:

- ▷ What is autograd in TensorFlow
- ▷ How to make use of autograd and an optimizer to solve an optimization problem

Let's get started.

Overview

This chapter is in three parts; they are:

- ▷ Autograd in TensorFlow
- ▷ Using Autograd for Polynomial Regression
- ▷ Using Autograd to Solve a Math Puzzle

3.1 Autograd in TensorFlow

In TensorFlow 2.x, you can define variables and constants as TensorFlow objects and build an expression with them. The expression is essentially a function of the variables. Hence you may derive its derivative function, i.e., the differentiation or the gradient. This feature is one of the many fundamental features in TensorFlow. The deep learning model will make use of this in the training loop.

It is easier to explain autograd with an example. In TensorFlow 2.x, you can create a constant matrix as follows:

```
import tensorflow as tf

x = tf.constant([1, 2, 3])
print(x)
print(x.shape)
print(x.dtype)
```

Listing 3.1: A constant matrix defined in TensorFlow

The above prints:

```
tf.Tensor([1 2 3], shape=(3,), dtype=int32)
(3,)
<dtype: 'int32'>
```

Output 3.1: Constant matrix created from Listing 3.1

This creates an integer vector (in the form of `Tensor` object). This vector can work like a NumPy vector in most cases. For example, you can do `x+x` or `2*x`, and the result is just what you would expect. TensorFlow comes with many functions for array manipulation that match NumPy, such as `tf.transpose` or `tf.concat`.

Creating variables in TensorFlow is just the same, for example:

```
import tensorflow as tf

x = tf.Variable([1, 2, 3])
print(x)
print(x.shape)
print(x.dtype)
```

Listing 3.2: A variable defined in TensorFlow

This will print:

```
<tf.Variable 'Variable:0' shape=(3,) dtype=int32, numpy=array([1, 2, 3], dtype=int32)>
(3,)
<dtype: 'int32'>
```

Output 3.2: Variable created from Listing 3.2

The operations (such as `x+x` and `2*x`) that you can apply to `Tensor` objects can also be applied to variables. The difference between variables and constants is that the former allows the value to change while the latter is immutable. This distinction is important when you run a *gradient tape* as follows:

```
import tensorflow as tf

x = tf.Variable(3.6)

with tf.GradientTape() as tape:
    y = x*x
```

```
dy = tape.gradient(y, x)
print(dy)
```

Listing 3.3: Getting gradient in TensorFlow

This prints:

```
tf.Tensor(7.2, shape=(), dtype=float32)
```

Output 3.3: Gradient as obtained from Listing 3.3

What it does is the following: This defined a variable `x` (with value 3.6) and then created a gradient tape. While the gradient tape is working, it computes `y=x*x` or $y = x^2$. The gradient tape monitored how the variables are manipulated. Afterward, you asked the gradient tape to find the derivative $\frac{dy}{dx}$. You know $y = x^2$ means $\frac{dy}{dx} = 2x$. Hence the output would give you a value of $3.6 \times 2 = 7.2$.

3.2 Using Autograd for Polynomial Regression

How this feature in TensorFlow helpful?

Let's consider a case where you have a polynomial in the form of $y = f(x)$, and you are given several (x, y) samples. How can you recover the polynomial $f(x)$? One way is to assume random coefficients for the polynomial and feed in the samples (x, y). If the polynomial is found, you should see the value of y matches $f(x)$. The closer they are, the closer your estimate is to the correct polynomial. This is indeed a numerical optimization problem such that you want to minimize the difference between y and $f(x)$. You can use gradient descent to solve it.

Let's consider an example. You can build a polynomial $f(x) = x^2 + 2x + 3$ in NumPy as follows:

```
import numpy as np

polynomial = np.poly1d([1, 2, 3])
print(polynomial)
```

Listing 3.4: Defining a polynomial in NumPy

This prints:

```
   2
1 x + 2 x + 3
```

Output 3.4: A polynomial created

You may use the polynomial as a function, such as:

```
print(polynomial(1.5))
```

Listing 3.5: Using a polynomial

And this prints `8.25`, for $(1.5)^2 + 2 \times (1.5) + 3 = 8.25$.

Now you can generate a number of samples from this function using NumPy:

```
N = 20   # number of samples

# Generate random samples between -10 to +10
X = np.random.uniform(-10, 10, size=(N,1))
Y = polynomial(X)
```

Listing 3.6: Making samples from a polynomial

In the above, both `X` and `Y` are NumPy arrays of the shape `(20,1)`, and they are related as $y = f(x)$ for the polynomial $f(x)$.

Now, assume you do not know what the polynomial is, except it is quadratic. And you want to recover the coefficients. Since a quadratic polynomial is in the form of $Ax^2 + Bx + C$, you have three unknowns to find. You can find them using the gradient descent algorithm you implement or an existing gradient descent optimizer. The following demonstrates how it works:

```
import tensorflow as tf

# Assume samples X and Y are prepared elsewhere

XX = np.hstack([X*X, X, np.ones_like(X)])

w = tf.Variable(tf.random.normal((3,1)))  # the 3 coefficients
x = tf.constant(XX, dtype=tf.float32)     # input sample
y = tf.constant(Y, dtype=tf.float32)      # output sample
optimizer = tf.keras.optimizers.Nadam(learning_rate=0.01)
print(w)

for _ in range(1000):
    with tf.GradientTape() as tape:
        y_pred = x @ w
        mse = tf.reduce_sum(tf.square(y - y_pred))
    grad = tape.gradient(mse, w)
    optimizer.apply_gradients([(grad, w)])

print(w)
```

Listing 3.7: Regression on samples to discover the polynomial

The `print` statement before the for loop gives three random number, such as

```
<tf.Variable 'Variable:0' shape=(3, 1) dtype=float32, numpy=
array([[-2.1450958 ],
       [-1.1278448 ],
       [ 0.31241694]], dtype=float32)>
```

Output 3.5: Vector `w` is three random numbers

But the one after the for loop gives you the coefficients very close to that in the polynomial:

```
<tf.Variable 'Variable:0' shape=(3, 1) dtype=float32, numpy=
array([[1.0000628],
       [2.0002015],
       [2.996219 ]], dtype=float32)>
```

Output 3.6: Vector `w` as the result of regression

What the above code does is the following: First, it creates a variable vector `w` of 3 values, namely the coefficients A, B, C. Then you create an array of shape $(N, 3)$, in which N is the number of samples in the array `X`. This array has 3 columns, which are the values of x^2, x, and 1, respectively. Such an array is built from the vector `X` using the `np.hstack()` function. Similarly, we build the TensorFlow constant `y` from the NumPy array `Y`. Afterwards, you use a for-loop to run gradient descent in 1,000 iterations. In each iteration, you compute $x \times w$ in matrix form to find $Ax^2 + Bx + C$ and assign it to the variable `y_pred`. Then, compare `y` and `y_pred` and find the mean square error. Next, derive the gradient, i.e., the rate of change of the mean square error with respect to the coefficients `w`. And based on this gradient, you use gradient descent to update `w`.

In essence, the above code is to find the coefficients `w` that minimizes the mean square error. Putting everything together, the following is the complete code:

```
import numpy as np
import tensorflow as tf

N = 20   # number of samples

# Generate random samples between -10 to +10
polynomial = np.poly1d([1, 2, 3])
X = np.random.uniform(-10, 10, size=(N,1))
Y = polynomial(X)

# Prepare input as an array of shape (N,3)
XX = np.hstack([X*X, X, np.ones_like(X)])

# Prepare TensorFlow objects
w = tf.Variable(tf.random.normal((3,1)))  # the 3 coefficients
x = tf.constant(XX, dtype=tf.float32)     # input sample
y = tf.constant(Y, dtype=tf.float32)      # output sample
optimizer = tf.keras.optimizers.Nadam(learning_rate=0.01)
print(w)

# Run optimizer
for _ in range(1000):
    with tf.GradientTape() as tape:
        y_pred = x @ w
        mse = tf.reduce_sum(tf.square(y - y_pred))
    grad = tape.gradient(mse, w)
    optimizer.apply_gradients([(grad, w)])

print(w)
```

Listing 3.8: Regression to discover a polynomial using TensorFlow

3.3 Using Autograd to Solve a Math Puzzle

In the above, 20 samples were used, which is more than enough to fit a quadratic equation. You may use gradient descent to solve some math puzzles as well. For example, the following problem:

$$\begin{array}{ccccc} A & + & B & = & 8 \\ + & & + & & \\ C & - & D & = & 6 \\ = & & = & & \\ 13 & & 8 & & \end{array}$$

In other words, to find the values of A, B, C, D such that:

$$A + B = 8$$

$$C - D = 6$$

$$A + C = 13$$

$$B + D = 8$$

This can also be solved using autograd, as follows:

```
import tensorflow as tf
import random

A = tf.Variable(random.random())
B = tf.Variable(random.random())
C = tf.Variable(random.random())
D = tf.Variable(random.random())

# Gradient descent loop
EPOCHS = 1000
optimizer = tf.keras.optimizers.Nadam(learning_rate=0.1)
for _ in range(EPOCHS):
    with tf.GradientTape() as tape:
        y1 = A + B - 8
        y2 = C - D - 6
        y3 = A + C - 13
        y4 = B + D - 8
        sqerr = y1*y1 + y2*y2 + y3*y3 + y4*y4
    gradA, gradB, gradC, gradD = tape.gradient(sqerr, [A, B, C, D])
    optimizer.apply_gradients([(gradA, A), (gradB, B), (gradC, C), (gradD, D)])

print(A)
print(B)
print(C)
print(D)
```

Listing 3.9: Solving a math puzzle using TensorFlow

There can be multiple solutions to this problem. One solution is the following:

```
<tf.Variable 'Variable:0' shape=() dtype=float32, numpy=3.500003>
<tf.Variable 'Variable:0' shape=() dtype=float32, numpy=4.5006905>
<tf.Variable 'Variable:0' shape=() dtype=float32, numpy=9.499581>
<tf.Variable 'Variable:0' shape=() dtype=float32, numpy=3.540172>
```

Output 3.7: Solution as found by Listing 3.9

Which means $A = 3.5$, $B = 4.5$, $C = 9.5$, and $D = 3.5$. You can verify this solution fits the problem.

The above code defines the four unknown as variables with random initial values. Then you compute the result of the four equations and compare it to the expected answer. You then sum up the squared error and ask TensorFlow to minimize it. The minimum possible square error is zero, attained when our solution exactly fits the problem.

Note the way the gradient tape is asked to produce the gradient: You ask the gradient of `sqerr` respective to `A`, `B`, `C`, and `D`. Hence four gradients are found. You then apply each gradient to the respective variables in each iteration. Rather than looking for the gradient in four different calls to `tape.gradient()`, this is required in TensorFlow because the gradient of `sqerr` can only be recalled once by default.

3.4 Further Reading

This section provides more resources on the topic if you are looking to go deeper.

Articles

Introduction to gradients and automatic differentiation. TensorFlow.
https://www.tensorflow.org/guide/autodiff

Advanced automatic differentiation. TensorFlow.
https://www.tensorflow.org/guide/advanced_autodiff

3.5 Summary

In this chapter, we demonstrated how TensorFlow's automatic differentiation works. This is the building block for carrying out deep learning training. Specifically, you learned:

- ▷ What is automatic differentiation in TensorFlow
- ▷ How you can use gradient tape to carry out automatic differentiation
- ▷ How you can use automatic differentiation to solve a optimization problem

In the next chapter, you will learn about the higher-level library for deep learning, Keras.

Introduction to Keras

4

TensorFlow is a very powerful library, but can be difficult to use directly for creating deep learning models. In this chapter, you will discover the Keras Python library that provides a clean and convenient way to create a range of deep learning models on top of TensorFlow. After completing this chapter, you will know:

- ▷ About the Keras Python library for deep learning.
- ▷ The standard idiom for creating models with Keras.

Let's get started.

Overview

This chapter is in three parts; they are:

- ▷ What is Keras?
- ▷ How to Install Keras
- ▷ Using Autograd to Solve a Math Puzzle

4.1 What is Keras?

Keras is a minimalist Python library for deep learning that can run on top of TensorFlow. It was developed to make implementing deep learning models as fast and easy as possible for research and development. It runs on Python and can seamlessly execute on GPUs and CPUs through TensorFlow. It is released under the permissive MIT license.

Keras was developed and maintained by François Chollet, a Google engineer, using four guiding principles:

- ▷ **Modularity**: A model can be understood as a sequence or a graph alone. All the concerns of a deep learning model are discrete components that can be combined in arbitrary ways.
- ▷ **Minimalism**: The library provides just enough to achieve an outcome, no frills and maximizing readability.

- ▷ **Extensibility**: New components are intentionally easy to add and use within the framework, intended for researchers to trial and explore new ideas.
- ▷ **Python**: No separate model files with custom file formats. Everything is native Python.

4.2 How to Install Keras

Keras is relatively straightforward to install if you already have a working Python environment. If you need help setting up your workstation, see Appendix B. In the past, Keras was a standalone library. Nowadays, it is part of TensorFlow. Therefore you just need to install TensorFlow using `pip`, as follows:

```
pip install tensorflow
```

For backward compatibility reason, Keras can also be installed as follows:

```
pip install keras
```

And functionally you installed the same library.

At the time of writing, the most recent version of TensorFlow is version 2.9.2. You can check your version of TensorFlow on the command line using the following snippet:

```
python -c "import tensorflow; print(tensorflow.__version__)"
```

Listing 4.1: Checking the version of TensorFlow installed

Running the above script you will see:

```
2.9.2
```

Output 4.1: TensorFlow version

You can upgrade your installation of TensorFlow/Keras using the same method:

```
pip install --upgrade tensorflow
```

4.3 Build Deep Learning Models with Keras

With TensorFlow, you can build a neural network by creating variables for the weights and define how they interact with the training data as input to produce outputs. You already saw how to use TensorFlow this way in Chapter 3. However, this is too tedious and inefficient. Using Keras helps you to think in terms of layers instead of individual weights and parameters.

Keras centers on the concept of a model. The main model type is a called a `Sequential` which is a linear stack of layers. You create a `Sequential` and add layers to it in the order that the computation should be performed. Once defined, you compile the model to make

use of the underlying framework to optimize computations to be performed. In this you can specify the loss function and the optimizer to be used.

Once compiled, the model must be fit to data. This can be done one batch of data at a time or by firing off the entire model training regime. This is where all the compute happens. Once trained, you can use your model to make predictions on new data. We can summarize the construction of deep learning models in Keras as follows:

1. **Define your model**. Create a `Sequential` model and add configured layers.
2. **Compile your model**. Specify loss function and optimizers and call the `compile()` function on the model.
3. **Fit your model**. Train the model on a sample of data by calling the `fit()` function on the model.
4. **Make predictions**. Use the model to generate predictions on new data by calling functions such as `evaluate()` or `predict()` on the model.

4.4 Keras Resources

The list below provides some additional resources that you can use to learn more about Keras.

tf.keras module. TensorFlow.
https://www.tensorflow.org/api_docs/python/tf/keras

Keras official homepage.
https://keras.io

Keras project on GitHub.
https://github.com/keras-team/keras

4.5 Summary

In this chapter, you discovered the Keras Python library for deep learning research and development. You learned:

- ▷ Keras is a high-level library for TensorFlow, abstracting their capabilities and hiding their complexity.
- ▷ Keras is designed for minimalism and modularity, allowing you to very quickly define deep learning models.
- ▷ Keras deep learning models can be developed using an idiom of defining, compiling and fitting models that can then be evaluated or used to make predictions.

Begining with next chapter, you will see how Keras can be used to build various deep learning models.